Final Report on Zero-Knowledge Proofs

Nithesh

1 Introduction

Over the course of this project, I explored the foundations of zero-knowledge proofs (ZKPs). At the start of the project, many of the concepts in ZKPs were very unfamiliar to me. One of my initial confusions was the difference between ZKP and encryption, since both of them seem to involve hiding information. I initially thought that they served similar purposes. However, I later learnt that they are totally different concepts. As, encryption hides a message so that someone with the correct key can later decrypt and recover it. In contrast, a ZKP allows a prover to convince a verifier that they know some secret information without revealing the secret itself.

Throughout this project, I was able to build my understanding of the foundations of ZKPs and gradually move on to more advanced concepts. I later read classic ZKP papers, including Groth16 and Spartan, and attempted to implement Groth16 to gain a practical understanding of how the protocol is constructed.

2 Foundations of ZKPs

At the beginning of the project, my mentor introduced me to a book on ZKPs that provided a broad survey of the fundamental concepts in the field. I started by learning about interactive proof systems, where there are two main parties: a prover and a verifier. The prover tries to convince the verifier that a particular statement is true, while the verifier checks the proof. In a ZKP, the prover can convince the verifier that they know a secret without revealing it. The prover and verifier may exchange several messages during this process, which forms the interactive proof system. One interesting idea that I learnt throughout ZKPs is the use of randomness to verify large computations efficiently, without having to do all the computation directly.

From there, I was introduced to the basic properties of proof systems, such as completeness and soundness, as well as the difference between proofs and arguments. I also gradually learnt the mathematical foundations required for ZKPs, including groups, fields, arithmetic circuits, Lagrange interpolation, R1CS and witnesses.

One main concept in ZKP was the Schwartz-Zippel Lemma, as it provides an important idea behind many polynomial-based proof systems. My understanding is that if a non-zero polynomial has a bounded degree, then the probability that it evaluates to zero at a randomly chosen point is small. This means that instead of checking a large polynomial identity completely, a verifier can evaluate it at a random point and still obtain a high level of confidence that the claimed identity is correct.

I later studied the sum-check protocol, which was one of the most interesting concepts I encountered. Initially, I found the notation and the different rounds of the protocol difficult to follow. However, I eventually understood the high-level idea that the verifier can verify without

manually doing all the sums. It was fascinating that it could be applied to the Boolean satisfiability problem (SAT) and even counting triangles in graphs. However, some of the polynomial transformations used to represent these problems were initially unintuitive to me.

I was also introduced to the Fiat-Shamir transformation, which shows how interactive protocols can be converted into non-interactive ones by generating the verifier's random challenges using a cryptographic hash function.

As I progressed, I learnt about polynomial commitments, where the main idea is to prevent the prover from changing a polynomial after committing to it. I later read the KZG polynomial commitment paper and the Bulletproofs paper. These provided my first exposure to how the basic mathematical concepts I had learnt could be combined to construct practical cryptographic proof systems.

3 First Programming Exercise

After learning the theory, my mentor gave me several programming tasks. One of them was to implement the Number Theoretic Transform (NTT) and its inverse, the INTT. Through this exercise, I learnt that NTT and INTT are commonly used for efficient polynomial multiplication, reducing the complexity from $O(n^2)$ to approximately $O(n \log n)$. I found this method initially quite counter-intuitive, especially how transforming the polynomials first could make multiplication significantly faster.

I was then tasked with implementing the KZG polynomial commitment scheme. This was more challenging because I initially did not understand how to translate the mathematical notation into code. My mentor suggested using the mcl cryptographic library, which was my first time working with it. Since it was very new to me, I had to spend time reading the documentation, understanding the notation and learning how to use the required functions.

Using the KZG implementation, I was then tasked to construct the Univariate Zero-Test Polynomial Interactive Oracle Proofs (PIOP) and Univariate Sum-Check PIOP. The Zero-Test is used to prove that a polynomial evaluates to zero over an entire set of points, while the sum-check proves that the sum of a polynomial's evaluations over a set is equal to a claimed value. These tasks were challenging but interesting, as they were my first experience combining polynomial operations, commitments and verification protocols into a working implementation.

4 Reading Classic ZKP Papers

After learning the fundamental concepts, my mentor gave me a list of important ZKP papers to explore. These included Groth16, PLONK, Spartan, HyperPlonk, Plookup, and Caulk. I briefly skimmed through these papers to understand their main ideas and the different directions of ZKP research. From this list, I chose Groth16 and Spartan to study in greater depth because they use very different approaches to construct efficient proof systems.

4.1 Groth16

This paper implements a pairing-based zkSNARK that is well known for having very small proof sizes and fast verification. The idea is the computation is first represented using R1CS, which is then converted into a Quadratic Arithmetic Program (QAP). The prover uses the QAP together with a trusted setup to generate a proof consisting of only a few elliptic-curve group elements, while the verifier checks the proof using bilinear pairings.

What I found interesting about Groth16 was how a potentially large computation could eventually be reduced into such a small proof. At the same time, understanding the protocol was

challenging because it combined many concepts I had learnt earlier and the mathematical notation was quite confusing at times.

After studying the paper, I attempted to implement it using the same library that I used for my first programming exercise. The implementation included constructing the R1CS, converting it into a QAP, generating the trusted setup, creating the proof and finally verifying it using pairings. One of the main difficulties during implementation was translating the equations from the paper into code. As I had to understand what each term represented and how it should be implemented using field and group operations.

4.2 Spartan

This paper implements a general-purpose zkSNARK, but it takes a very different approach from Groth16. Instead of converting R1CS into a QAP, it works more directly with the R1CS representation using multilinear extensions and the sum-check protocol. Depending on the polynomial commitment scheme used, Spartan does not need a trusted setup.

I found Spartan particularly interesting because many concepts that I had studied earlier, especially the sum-check protocol and polynomial commitments, appeared again as important parts of a proof system. It also introduced Spark, a technique used to efficiently handle the sparse matrices found in R1CS.

5 Challenges

The main challenge I faced throughout the project was reading and understanding the theoretical concepts. The book contained a large amount of information and used mathematical notation that was often difficult for me to follow at first. Because of this, I often referred to other resources online, including an MIT youtube lecture on ZKPs, which explained some of the concepts in a more intuitive way and helped me build a clearer understanding.

Some topics, such as the sum-check protocol, QAPs and polynomial commitment schemes, were especially difficult to understand immediately. For these, simply reading the definitions was not enough. I had to work through small examples, try different inputs and trace the steps manually before the ideas started to make sense.

Another challenge was moving from theory to implementation. Mathematical expressions in papers can look compact, but implementing them requires understanding exactly how field elements, polynomials, group elements and pairing operations are represented in code.

Another challenge was to use the mcl cryptographic library, which was unfamiliar to me at the start. Reading the documentation, testing smaller components separately and debugging each stage one by one helped me gradually overcome these implementation difficulties.

Overall, the challenges changed the way I approached difficult material. Even when I could not understand or solve something immediately, I learnt to break the problem down, trace it step by step, and work through each part until I understood it.

6 Conclusion

Overall, this project greatly improved my understanding of ZKPs, from the basic concepts to more advanced systems such as Groth16 and Spartan. Although I am still very new to this field, I now have a much better understanding of how interactive proof techniques work. I believe these ideas could potentially be applied to open problems in other fields as well, and exploring such applications may lead to useful and interesting results in the future.